MACHINE LEARNING

REPORT: Multi-Class Classification

Predictive Maintenance: Industrial Machine Condition

GitHub: [add your repository link here]

Items in square brackets, like [XX.XX], are values from your notebook output. Replace them with your actual numbers before submitting.

1. Introduction

The goal of this project is to build a machine learning pipeline that identifies the operating condition of an industrial machine from its sensor measurements. The dataset is synthetic but inspired by real predictive maintenance scenarios. Each machine belongs to one of three classes:

- **0 - Normal**
- **1 - Thermal Failure:** linked to excessive thermal stress and unfavourable temperature or cooling conditions
- **2 - Mechanical Failure:** linked to mechanical load, component wear, torque, speed and vibration

Models are evaluated on **classification accuracy** on a hidden test set.

This project includes:

- Data inspection and missing-value handling
- Exploratory data analysis (EDA), including outlier and correlation analysis
- Feature ranking using mutual information and feature selection
- A baseline comparison of five supervised models using 5-fold cross-validation
- Hyperparameter tuning of the best model with GridSearchCV
- Final model training and generation of the submission file

2. Dataset Overview

Three files were provided:

1. **Training file** - labelled machine records with the target column `condition`
2. **test.csv** - unlabelled records to be predicted
3. **sample_submission.csv** - required submission format (`unit_id`, `condition`)

Train shape: [rows x columns]. Test shape: [rows x columns].

2.1 Target Variable

`condition` has three classes. The class distribution is controlled and known:

Class	Meaning	Share
0	Normal	~50% (8,000 machines)
1	Thermal Failure	~25% (4,000 machines)
2	Mechanical Failure	~25% (4,000 machines)

The classes are moderately imbalanced (Normal is twice as large as each failure class), but nowhere near the severe imbalance often seen in failure data. A model that always predicts "Normal" would score about 50%, which is used as the baseline for comparison.

2.2 Input Features

The model uses 11 features:

- **Categorical / ordinal:** `unit_type` (S, M, H)
- **Environmental and thermal:** `ambient_temp`, `process_temp`, `coolant_flow`, `thermal_stress`
- **Mechanical:** `shaft_speed`, `drive_torque`, `power_load`, `mechanical_load`, `component_wear`, `vibration_index`

`unit_id` is an identifier and was excluded from modelling.

3. Preprocessing Pipeline

3.1 Encoding of the Categorical Feature

`unit_type` (S, M, H) was label-encoded as **S = 0, M = 1, H = 2**. This keeps a natural ordering (Small, Medium, Heavy) and adds only one column, which suits tree-based models.

3.2 Handling Missing Values

Missing values were counted in both the train and test sets. They were filled with the **mean of each feature**, computed on the training data only and applied to both train and test, so no test information leaks into training. After imputation, `unit_type` was rounded and clipped back to valid integer codes (0, 1, 2). A final check confirmed zero missing values remained in either set.

3.3 Feature Scaling

Scaling was applied only where it matters. **Logistic Regression** is trained inside a pipeline with `StandardScaler`, so the scaler is fitted on the training folds only. **Naive Bayes, Decision Tree, Random Forest and XGBoost** do not depend on feature scale and use unscaled values.

4. Exploratory Data Analysis (EDA)

Key analyses carried out:

- **Target distribution:** confirmed the 50 / 25 / 25 class split.
- **Feature distributions:** histograms for every numeric feature to check shape and skew.

- **Outlier detection (IQR method):** values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR were flagged. For each feature the share of outliers belonging to a failure class was also computed. **Decision: outliers were kept**, because extreme sensor readings can be genuine signs of machine failure.
- **Features vs condition:** overlaid density histograms, boxplots and per-class medians showed which sensors separate the three conditions. A table of condition percentages within each `unit_type` showed whether machine type affects failure rate.
- **Correlation analysis:** a full correlation heatmap and a list of feature pairs with |r| > 0.8 were produced to find redundant features.
- **Mutual information (MI) ranking:** features were ranked by how much information they carry about `condition`. MI captures non-linear relationships, which a simple mean comparison (ANOVA) would miss.

Key observations: [summarise 2-4 findings from your plots, for example which features separate Thermal vs Mechanical failures best, which pairs were strongly correlated, and the top 3 features by MI].

5. Feature Selection

The correlation analysis identified redundancy among the power-related features. The derived feature `mech_power` was **removed** because it was largely explained by other features (shaft speed and torque). The final feature set has **11 features**: `unit_type`, `ambient_temp`, `process_temp`, `coolant_flow`, `shaft_speed`, `drive_torque`, `power_load`, `component_wear`, `thermal_stress`, `mechanical_load`, `vibration_index`.

Two ratio features, `wear_torque` and `stress_per_flow`, were also considered: [state whether they were kept or dropped, and the CV accuracy with and without them].

A check with XGBoost confirmed the chosen feature set gives a CV accuracy of [XX.XX]%.

6. Validation Strategy

Instead of a single train/validation split, **5-fold cross-validation** was used (`KFold`, shuffled, `random_state = 42`). Each fold uses 80% of the data for training and 20% for validation, and every row is used for validation exactly once.

One `KFold` object is shared by **all** models and by the tuning step, so every model is tested on exactly the same splits. This makes the comparison fair. The class mix in each validation fold was printed and was close to the expected 50 / 25 / 25.

For each model three quantities were recorded:

- **Train accuracy:** how well the model fits data it has seen
- **Validation accuracy:** how well it predicts unseen data (the number that matters)
- **Overfit gap:** train accuracy minus validation accuracy; a large gap means memorisation

7. Supervised Models (Baseline Comparison)

Five models were trained with default settings:

Model	Train Acc	Val Acc	Val Std	Overfit Gap
Logistic Regression	[ ]	[ ]	[ ]	[ ]
Naive Bayes	[ ]	[ ]	[ ]	[ ]
Decision Tree	[ ]	[ ]	[ ]	[ ]
Random Forest	[ ]	[ ]	[ ]	[ ]
XGBoost	[ ]	[ ]	[ ]	[ ]

7.1 Logistic Regression (Linear Baseline)

A simple, interpretable baseline, trained with scaling inside a pipeline. Val accuracy: [ ]. [Comment: how it compares with the tree models and what that says about linear separability.]

7.2 Naive Bayes

Gaussian Naive Bayes assumes features are independent given the class. Since several sensors are correlated, this assumption is violated. Val accuracy: [ ].

7.3 Decision Tree

A single tree captures non-linear rules but tends to overfit. Train accuracy: [ ], Val accuracy: [ ], gap: [ ].

7.4 Random Forest

An ensemble of decorrelated trees that reduces the variance of a single tree. Val accuracy: [ ].

7.5 XGBoost

Gradient-boosted trees, which build trees sequentially to correct previous errors. XGBoost was the strongest default model with Val accuracy [ ]. [Confirm this against your table.]

8. Hyperparameter Tuning (XGBoost)

The best baseline model, XGBoost, was tuned with `GridSearchCV` using the same 5-fold splits and accuracy as the metric.

Hyperparameter	Values searched
`n_estimators`	100, 300
`max_depth`	3, 5, 7
`learning_rate`	0.05, 0.1

This gives 12 combinations, or 60 model fits in total.

Best parameters: [n_estimators = , max_depth = , learning_rate =]

	CV Accuracy
Default XGBoost	[ ]
Tuned XGBoost	[ ]
Improvement	[ ]

Fold-by-fold accuracies of the tuned model: [ ], with mean [ ] and standard deviation [ ]. A low standard deviation shows the model performs consistently across folds.

9. Final Model

The final model is the **tuned XGBoost classifier** with the best parameters from the grid search, trained on the **entire training set** ([N] rows, 11 features). It uses `random_state = 42` and `eval_metric = "mlogloss"`.

9.1 Feature Importance

XGBoost feature importance was plotted for the final model. The most important features were [list the top 3-5]. [Link this to the physical meaning: for example, thermal features driving Thermal Failure and torque, wear and vibration driving Mechanical Failure.]

10. Kaggle Submission

Predictions were generated for the test set using the final model. The submission was built to match `sample_submission.csv` exactly, with these automated checks:

- Row count equals the sample submission
- `unit_id` order is identical to the sample submission
- No missing predictions
- All predictions are valid classes (0, 1 or 2)

The predicted class distribution on the test set was [ ]% / [ ]% / [ ]% for Normal / Thermal / Mechanical, which should be close to the training distribution of 50 / 25 / 25.

Final Kaggle score (accuracy): [ ]

11. Key Insights & Discussion

1. Tree-based ensemble models clearly outperformed the linear and probabilistic baselines, indicating non-linear relationships between sensors and machine condition.
2. Keeping outliers was justified: extreme sensor readings are often genuine failure signals rather than noise.
3. Removing redundant features (`mech_power`) simplified the model without hurting accuracy.
4. Using one shared set of CV folds made the model comparison fair and reliable.
5. Hyperparameter tuning improved XGBoost by [ ] percentage points, a modest gain over a strong default.

6 [Add any finding about which class is hardest to separate, for example from the confusion matrix or classification report.]

12. Limitations

- The data is synthetic, so real-world sensor noise, drift and rare failure modes may not be fully represented.
- The hyperparameter grid was small (12 combinations); a wider or randomised search might find better settings.
- Accuracy hides per-class performance; a failure class could be predicted worse than Normal even when overall accuracy is high.
- Missing values were filled with simple mean imputation, which ignores relationships between features.

13. Future Work

- Try LightGBM and CatBoost, and compare with XGBoost
- Wider hyperparameter search (random search or Bayesian optimisation)
- Domain-driven feature engineering (for example heat-per-flow, wear-per-load ratios)
- SHAP analysis for deeper model explanation
- Stacking or blending several strong models
- Evaluate per-class precision, recall and F1 using a confusion matrix
- Stratified cross-validation to keep the class mix identical across folds

14. Conclusion

This project built a complete machine learning pipeline for classifying industrial machine condition into Normal, Thermal Failure and Mechanical Failure. The work covered data cleaning, EDA, outlier and correlation analysis, mutual-information feature ranking, feature selection, a five-model baseline comparison under 5-fold cross-validation, hyperparameter tuning and submission generation.

The final tuned XGBoost model achieved a cross-validated accuracy of about [ ]% and a Kaggle score of [ ], well above the 50% majority-class baseline.